ARCHITECTURE

Four packages, one honest stack

pi-coding-agent

the product: CLI, TUI, sessions, extensions, skills, modes

pi-agent-core

the loop: agent state, tool execution, event stream

pi-ai

unified LLM API: providers, credentials, streaming

pi-tui standalone terminal UI — differential rendering

All TypeScript, all MIT, all on npm under @earendil-works. The layers are real: take pi-ai alone as a provider-agnostic LLM client, run the loop headless without a UI, or use the TUI for something that isn't an agent at all.

→ **15+ providers**

Anthropic, OpenAI, Google, Azure, Bedrock, Mistral, Groq, OpenRouter, local llama.cpp… switch mid-session.

→ **no flicker, no bloat**

The TUI renders differentially and holds very large sessions in a few hundred kilobytes.

→ **one core everywhere**

TUI, headless modes, and SDK all drive the exact same agent — no forked behaviors.

EXTENSIBILITY

Everything else is an extension

An extension is a TypeScript module loaded with jiti — no build step, no manifest, no SDK ceremony. Export a function, get the full API.

`~/.pi/agent/extensions/`	global — every session
`.pi/extensions/`	per-project — trusted repos only
`pi -e ./exp.ts`	one-off — for quick iteration

Tight loop: edit the file, reload from inside the session, try again — extension development happens while the agent runs, often with the agent writing the extension.

hello.ts

```ts
// ~/.pi/agent/extensions/hello.ts
import type { ExtensionAPI }
  from "@earendil-works/pi-coding-agent";

export default function (pi: ExtensionAPI) {
  pi.on("session_start", async (_ev, ctx) => {
    ctx.ui.notify("hello from an extension", "info");
  });
}

// that's a complete, working extension.
// no compile step - jiti loads .ts directly.
```

One API over the whole loop

extension api

```
// a permission gate in nine lines
pi.on("tool_call", async (ev) => {
  if (ev.toolName === "bash" &&
      ev.input.command.includes("rm -rf")) {
    return { block: true, reason: "not today" };
  }
});

// a slash command
pi.registerCommand("stats", {
  description: "Show statistics",
  handler: async (args, ctx) =>
    ctx.ui.notify(`Stats: ${args}`, "info"),
});
```

What you can hook

events

the full lifecycle — tool_call can block, tool_result can rewrite, input can transform, before_provider_request sees every payload

tools & commands

typed custom tools with streaming progress; slash commands; keybindings

ui

confirm / select / input dialogs, notifications, status bar, custom TUI components

state

append custom entries to the session — extension state survives resume and branching

models

register providers, switch model or thinking level programmatically

EXTENSIONS IN PRACTICE

What people actually build

Permission gates

Confirm dialogs for writes outside the repo; block dangerous commands outright.

+ Git checkpointing

Auto-commit or stash at every turn — rewind the code alongside the conversation.

× Protected paths

Refuse edits to .env, lockfiles, or prod configs, with a UI override when you mean it.

~ Custom compaction

Domain-specific summarizers that know what matters in your codebase.

> The omitted features

Sub-agents, plan mode, an MCP bridge — all exist as opt-in examples, not defaults.

$ Whole experiences

Wizards, dashboards, even games — the TUI component API is fully open.

50+ examples ship with the project — and most started life as “pi, write yourself an extension that…”

Skills: knowledge that loads itself

SKILL.md

```
skills/changelog/SKILL.md
---
name: changelog
description: Update CHANGELOG.md from
  merged PRs. Use when cutting a release
  or summarizing recent changes.
---

# Changelog workflow
1. List merges since the last tag
2. Group by area; keep entries terse
3. Link PRs; never invent changes
```

1. At startup, pi scans skill folders and extracts names + descriptions only.
2. Those one-liners ride along in the system prompt — a few tokens per skill.
3. When a task matches, the agent reads the full SKILL.md on demand.

Progressive disclosure — the same principle as CLI-tools-over-MCP, applied to knowledge. Costs stay near zero until the moment of use.

Loads from ~/.pi/agent/skills, .pi/skills, packages — and the shared ~/.agents/skills convention other tools read too.

DISTRIBUTION

Ship your whole setup as a package

A pi package bundles the four resource types — extensions (code), skills (knowledge), prompt templates, themes — and installs from npm, git, or a local path.

extensions/	behavior — gates, tools, commands
skills/	knowledge — workflows, conventions
prompts/	reusable prompt templates
themes/	how the TUI looks

The team angle: your org's permission gates, house skills, and review commands become one `pi install` for every engineer.

packages

```
$ pi install npm:@corp/pi-standards
$ pi install git:github.com/you/setup@v1
$ pi install ./packages/local-tools
$ pi list

// package.json — declare what you ship
{
  "pi": {
    "extensions": ["./extensions"],
    "skills":     ["./skills"]
  }
}

// tag it "pi-package" on npm ->
// it appears in the pi.dev gallery
```

THE PAYOFF

The agent that extends itself

> *“If you want the agent to do something that it doesn’t do yet, you don’t go and download an extension… you ask the agent to extend itself.”*
>
> — Armin Ronacher, running pi as his daily driver

A working setup that grew this way — commands and skills written and maintained by the agent:

`/answer` `/todos` `/review` `/control` `/files` `browser-via-CDP skill`

`commit-message skill` `changelog skill`

This is the compounding loop the tiny core buys: **a minimal, fully programmable harness makes the agent its own plugin author — software that builds more software.**

DAILY DRIVING

It’s also just a good place to work

/model, mid-session

Hop between 15+ providers without losing the thread. sonnet:high sets model and thinking in one flag; llama.cpp runs local.

a thinking dial

Reasoning effort from off to max, adjustable per message — pay for depth only when the task needs it.

queue while it works

Enter queues a steering note mid-run; Alt+Enter queues a follow-up for after; Esc aborts.

! and !!

Run shell yourself: ! shares the output with the model, !! keeps it private. You stay in the loop, literally.

@files and images

Tab-complete paths into the prompt; paste screenshots straight into context.

an honest footer

Live token count, cost, and current model always on screen — the meter never hides.

“It doesn’t flicker, it doesn’t consume a lot of memory, it doesn’t randomly break.” — Armin Ronacher

Headless: pi as infrastructure

`--mode print`

One shot: prompt in, answer out, exit. Scripts, cron, CI.

`--mode json`

Every agent event streamed as JSON lines. Logging, observability, replay.

`--mode rpc`

Full-duplex JSONL over stdin/stdout. Drive pi from any language.

rpc mode

```
$ pi --mode rpc
→ {"type":"prompt","message":"fix the failing test"}
← {"type":"response", ...}
← {"type":"tool_execution_start","tool":"bash", ...}
← {"type":"message_update", ...}
← {"type":"agent_end", ...}

commands: prompt · bash · set_model · get_state · compact · abort
```

IDE plugins, Slack bots, review pipelines, fleets of agents — all speak the same protocol the TUI uses.

And because sessions are plain JSONL, whatever you build reads exactly what the terminal user sees.

Or embed the loop directly

sdk

```
import { createAgentSession, ModelRuntime,
         SessionManager } from
  "@earendil-works/pi-coding-agent";

const { session } = await createAgentSession({
  sessionManager: SessionManager.inMemory(),
  modelRuntime: await ModelRuntime.create(),
});

session.subscribe((ev) => {
  if (ev.type === "message_update") render(ev);
});

await session.prompt("What files are in this repo?");
```

`the CLI is a client`

The terminal app is a thin layer over this same SDK — nothing it does is privileged.

`defineTool()`

Typed custom tools; sessions in memory or persisted as trees.

`extensions carry over`

The same extension files run in embedded mode — one plugin model everywhere.

Build your own product on the identical core — **harness as a library.**

SECURITY

Security: honesty over theater

pi defaults to full access and says so out loud. The write-up's blunt framing:

> *"As soon as your agent can write code and run code, it's pretty much game over."*

Per-command approval prompts on a code-writing, code-running agent are vibes, not a boundary — a capable model routes around them without even meaning to.

Real isolation is the OS's job

Run pi inside a container or VM — the docs ship a containerization guide. That's a boundary that actually holds.

Your gates, your code

Where friction helps, write it: tool_call interceptors that block, confirm, or rewrite — policy you can read and version.

Trust as a boundary

Project-local extensions and skills load only for directories you've explicitly trusted.

EVIDENCE

Does the minimal harness cost you?

No — and that’s the finding worth sitting with.

If four tools and a one-paragraph prompt keep pace with the most heavily engineered harnesses in the industry, then most harness complexity isn’t where capability comes from. The model is. What the harness really decides is how many tokens you waste — and how much of the work you can see.

So spend your context on the task — not on the scaffolding.

Terminal-Bench 2.0

Running Claude Opus, pi scores competitively with the heavyweight harnesses — Codex, Cursor, Windsurf — on the standard agentic terminal benchmark.

The Terminus 2 datapoint

A reference agent that is little more than raw tmux interaction “held its own against agents with far more sophisticated tooling.” The harness was never the moat.

TAKEAWAYS

Steal these ideas — whatever you run

1 **Count your context.** Know what every resident token is doing for you. Audit your harness the way pi audits itself.

2 **Prefer files to features.** Plans, todos, and knowledge as artifacts in the repo — durable, diffable, shareable.

3 **Make capability discoverable, not resident.** CLI tools and skills that cost nothing until the moment of use.

4 **Demand a programmable harness.** Transcripts you can parse, prompts you can read, a loop you can hook.

5 **Let the agent build its own tooling.** The compounding loop: an agent that writes its extensions gets better every week.

---

pi.dev · docs: pi.dev/docs · npm: @earendil-works/pi-coding-agent · MIT

Sources: the pi documentation (pi.dev/docs) · Mario Zechner, “What I learned building an opinionated and minimal coding agent” (mariozechner.at) · Armin Ronacher’s field notes (lucumr.pocoo.org)



Hermes Agent v0.18.0 (2026.7.1) · upstream c5f5fa40



argo:claude-opus-4.8 · Nous Research
/Users/stevens
Session: 20260730_193821_1fff3e

Available Tools

browser: browser_back, browser_click, ...
clarify: clarify
code_execution: execute_code
computer_use: computer_use
cronjob: cronjob
delegation: delegate_task
file: patch, read_file, search_files, write_file
image_gen: image_generate
(and 11 more toolsets...)

MCP Servers

ocr (stdio) — 8 tool(s)
ump (stdio) — connecting
arca (http) — 8 tool(s)

Available Skills

apple: apple-notes, apple-reminders, findmy, imessage,...
autonomous-ai-agents: cc-ask-headless, claude-code, codex, cx-gm-ask-...
creative: architecture-diagram, ascii-art, ascii-video, b...
data-science: affiliation-entity-resolution, corpus-structure...
devops: alcf-corpus-pullback, argo-proxy-fleet-access, ...
email: himalaya
gaming: minecraft-modpack-server, pokemon-player
general: computer-use, dogfood, yuanbao
github: codebase-inspection, github-auth, github-code-r...
hpc: alcf-batch-jobs, crux, intel-oneapi-gpu-toolchain
mcp: build-mcp-service, native-mcp
media: gif-search, heartmula, songsee, spotify, youtub...
mlops: audiocraft-audio-generation, build-mcp-service-...
mlops-evaluation: benchbench, benchmark-eval-resume-discipline
note-taking: obsidian
productivity: airtable, document-to-editable-latex, google-wo...
red-teaming: godmode
research: academic-pdf-harvest, arxiv, blogwatcher, conte...
smart-home: openhue
social-media: xurl
software-development: agent-sdk-loop-mapping, debugging-hermes-tui-co...

46 tools · 148 skills · 2 MCP servers · /help for commands
⚠ 6803 commits behind — run hermes update to update

↻ Resumed session 20260730_193821_1fff3e (2 user messages, 3 total messages)
↻ Working directory: /Users/stevens/Dropbox/AI-ENVIRONMENT

Previous Conversation

● You: Reply with exactly HERMES_ARGO_OK
◆ Hermes: HERMES_ARGO_OK
● You: can you look at the LoopX repo on github

Kukla online — Argo/Opus-4.8 via CherryRd. /help for commands.
+ Tip: Each profile gets its own config, API keys, memory, sessions, skills, and cron jobs.

argo:claude-opus-4.8 | 1fff3e | ctx -- | [   ] -- | 3s | 0s

◆

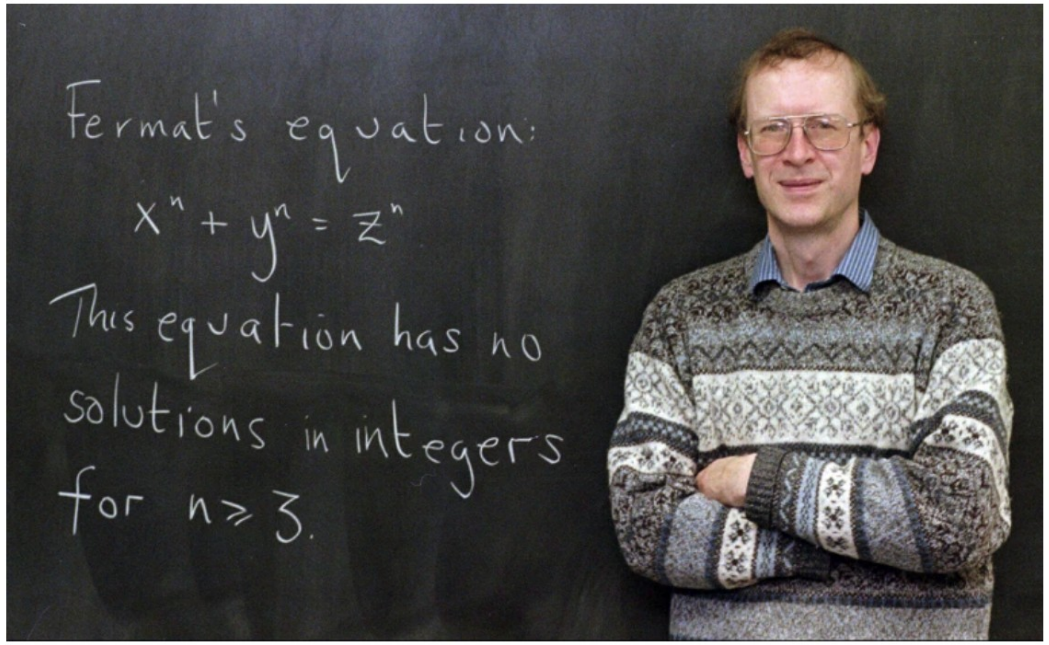
AI Highlights

Anthropic AI ‘formalizes’ proof of Fermat’s last theorem in just 11 days

Claude produced a 13-million-line, computer-checked proof of the famed conjecture — a major milestone in mathematics.

By Davide Castelvecchi

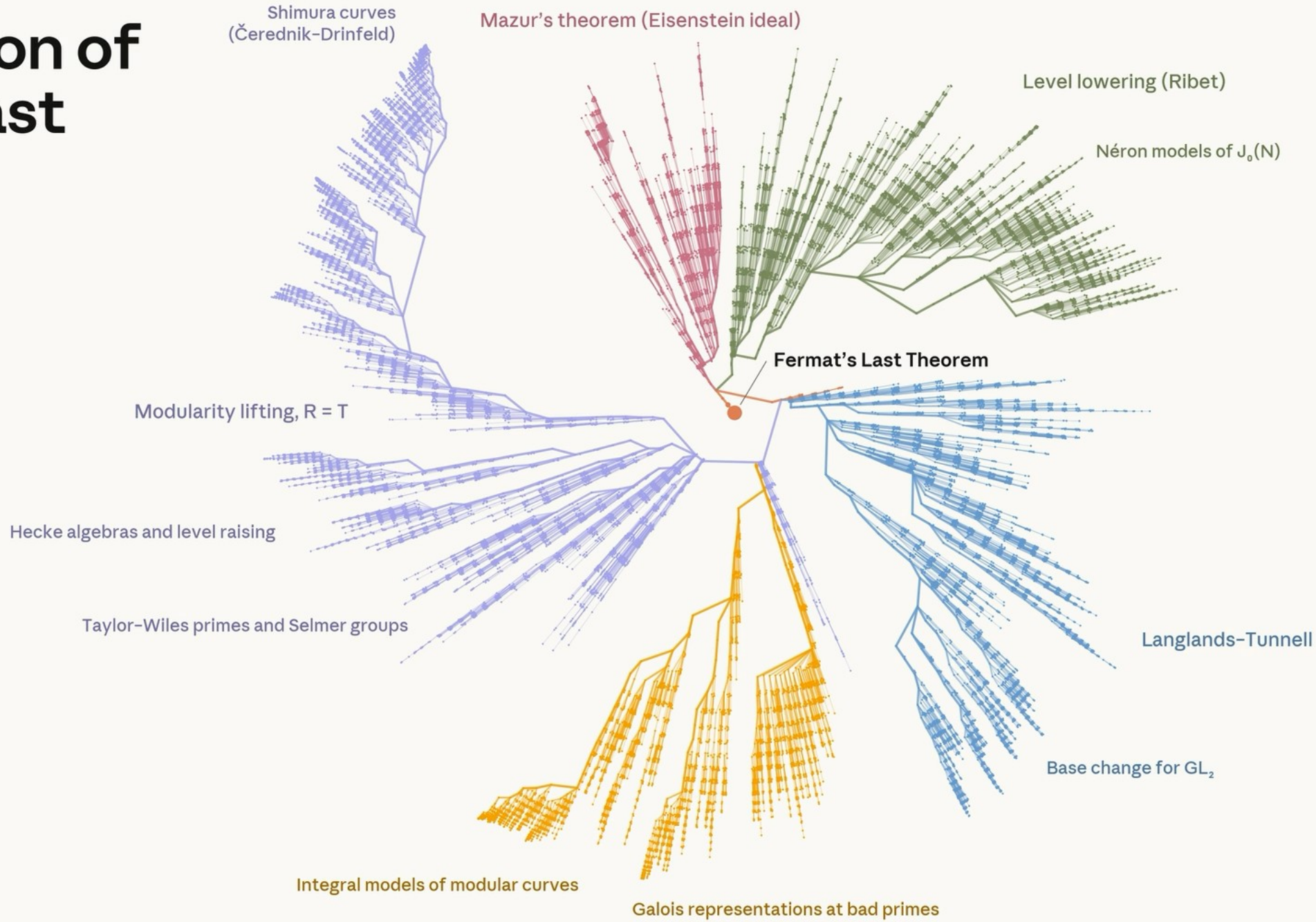
Save article

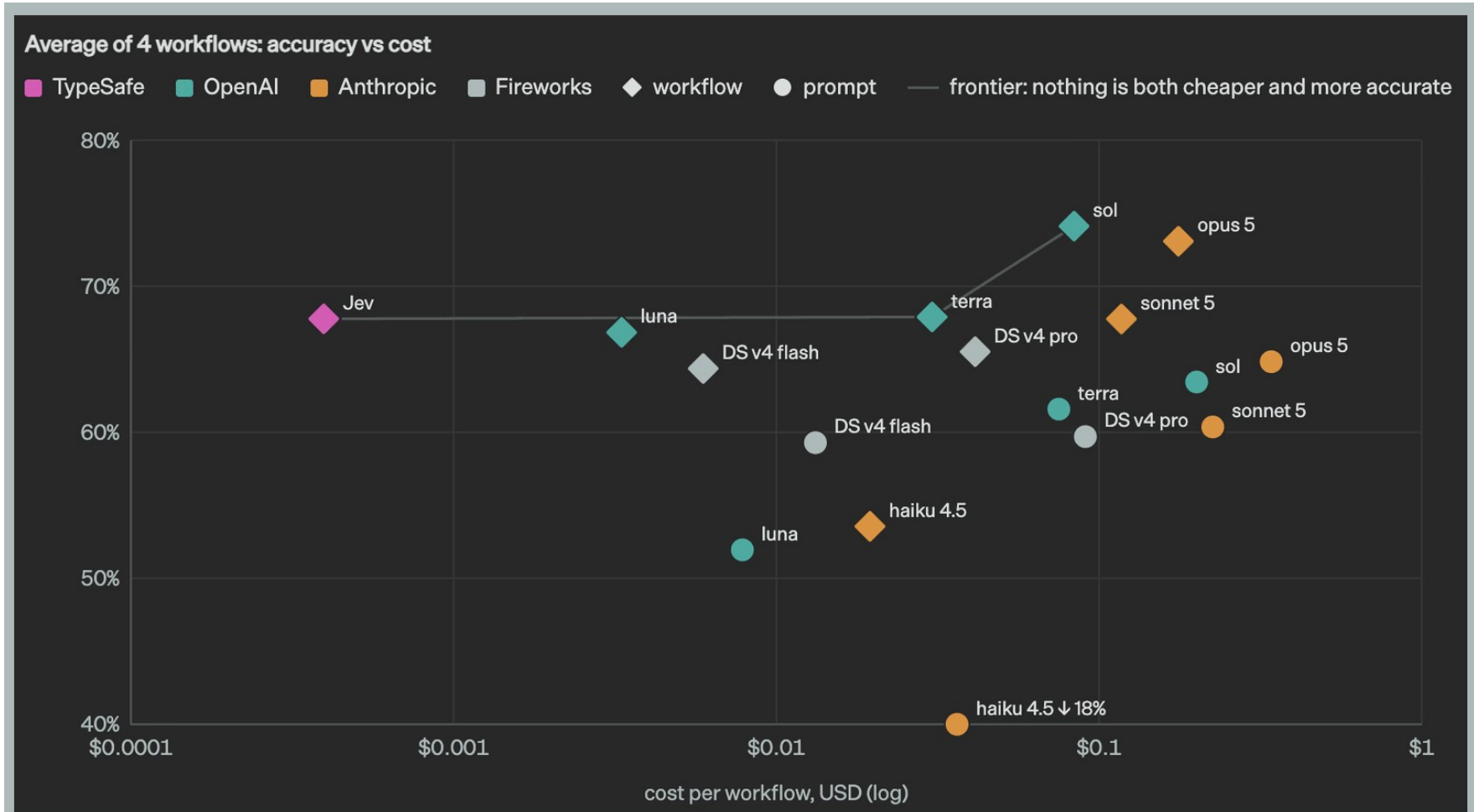


Mathematician Andrew Wiles completed the proof to Fermat’s last theorem in 1994, more than 350 years after Pierre Fermat proposed the conjecture. Credit: AP Photo/Charles Rex Arbogast/Alamy

Fermat’s last theorem, one of the most celebrated mathematical results of the last half-century, has been turned into computer-verified code for the first time, using an advanced prototype of the artificial-intelligence (AI) chatbot Claude.

Formalization of Fermat's Last Theorem





Jev is off the charts - owning the Pareto frontier for almost 2 orders of magnitude. We also compare to models with a generated prompt doing all the logic in their chain-of-thought, but this tends to do significantly worse than using the workflow itself.

Typesafe.
ai

OpenAI's GPT-6 Astra on ARC-AGI-3

Summary

- GPT-6 Astra scores 62.7% for $26K on ARC-AGI-3 Semi-Private with our Standard harness, and 99.9% for $19K with a Provider Adapter harness.
- GPT-6 Astra surpasses the human baseline in action efficiency on ARC-AGI-3. It used fewer actions than the median tested human on 96% of levels.
- A key behavior observed in GPT-6 Astra was its ability to turn unfamiliar environments into compact symbolic world models. It represented game mechanics as logical rules and developed its own domain-specific language shorthand to track state and plan actions.

ARC-AGI-3

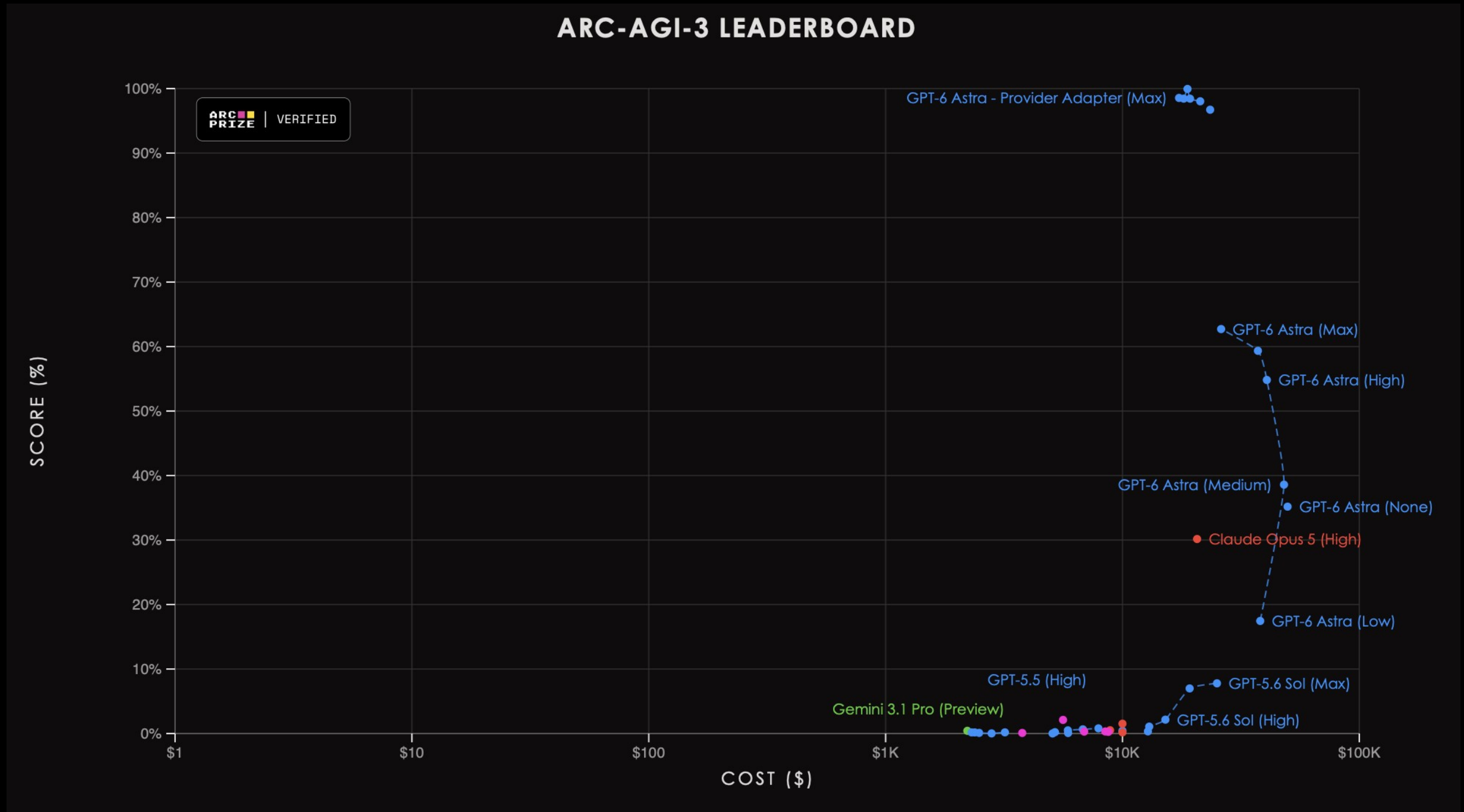
ARC-AGI-3 is a benchmark for studying agentic intelligence through novel, abstract, turn-based environments. Agents must explore, infer goals, and build internal models of environments to effectively plan actions *without* explicit instructions. You can play ARC-AGI-3 yourself.

OpenAI's GPT-6 Astra on ARC-AGI-3

A key behavior observed in GPT-6 Astra was its ability to turn unfamiliar environments into compact symbolic world models. It represented game mechanics as logical rules and developed its own domain-specific language shorthand to track state and plan actions.

ARC-AGI-3 is a benchmark for studying agentic intelligence through novel, abstract, turn-based environments. Agents must explore, infer goals, and build internal models of environments to effectively plan actions *without* explicit instructions. You can play ARC-AGI-3 yourself.

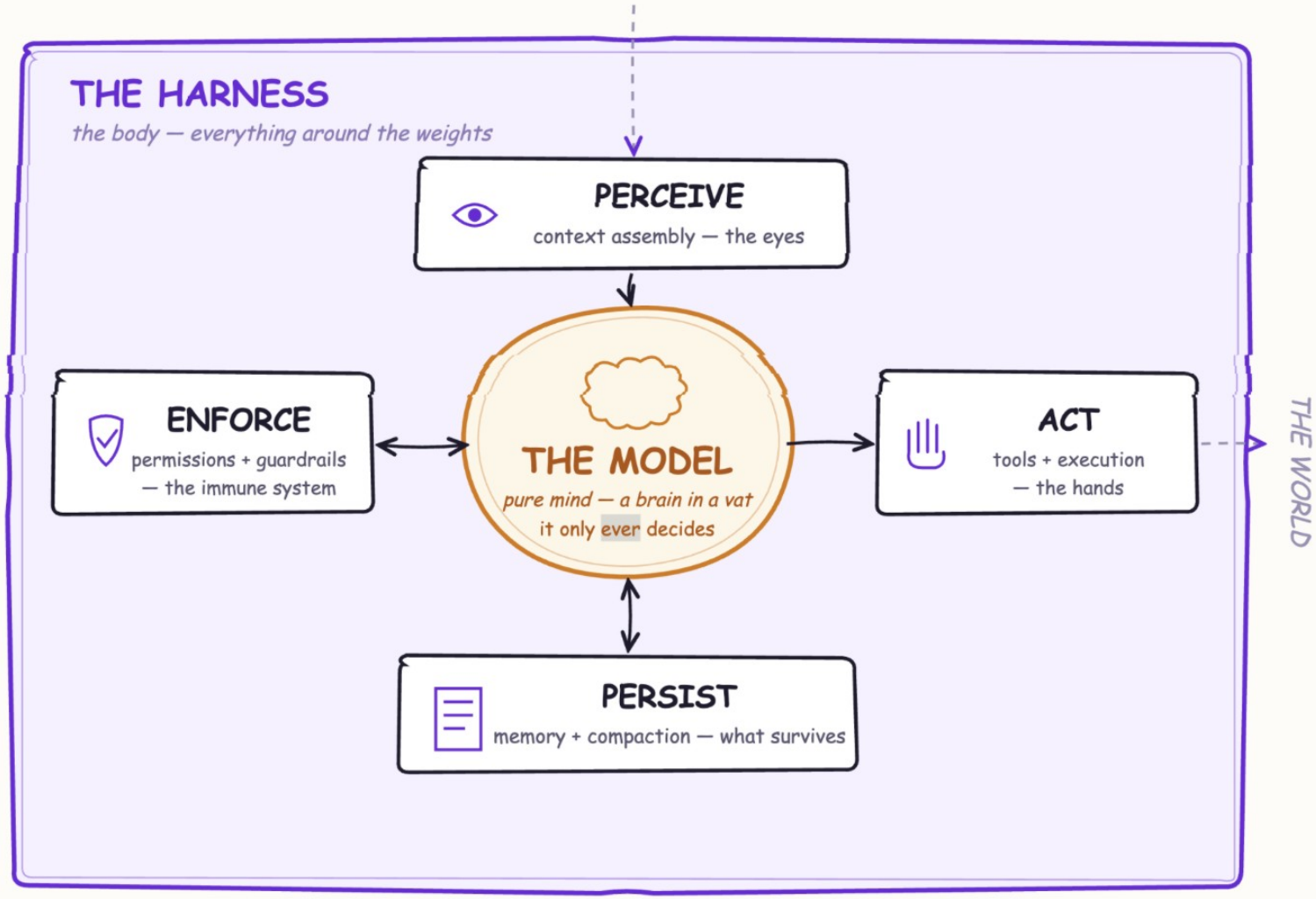

ls20
LEVEL 2 / 7
SPACEBAR
CLICK
UNDO (Z)
RESET
HELP
SELECT


ARC-AGI-3 LEADERBOARD
ARC PRIZE | VERIFIED
SCORE (%)
100%
90%
80%
70%
60%
50%
40%
30%
20%
10%
0%
$1
$10
$100
$1K
$10K
$100K
COST ($)
GPT-6 Astra - Provider Adapter (Max)
GPT-6 Astra (Max)
GPT-6 Astra (High)
GPT-6 Astra (Medium)
GPT-6 Astra (None)
Claude Opus 5 (High)
GPT-6 Astra (Low)
GPT-5.5 (High)
GPT-5.6 Sol (Max)
Gemini 3.1 Pro (Preview)
GPT-5.6 Sol (High)


ARC PRIZE | VERIFIED
SCORE (%)
100%
90%
80%
70%
60%
50%
40%
30%
20%
10%
0%
$1
$100K
(Max)
GPT-6 Astra (Max)
GPT-6 Astra (High)
PT-6 Astra (Medium)
GPT-6 Astra (None)
Claude Opus 5 (High)
GPT-6 Astra (Low)
GPT-5.6 Sol (Max)
GPT-5.6 Sol (High)
"GPT-6 Astra, trained on ~100K+ NVIDIA Grace Blackwell NVLink72.
From ChatGPT to o1 to Astra in 4 years.
AGI has arrived.
Congratulations @OpenAI team.
400K GPUs coming online next."
Jensen Huang
Founder & CEO, NVIDIA
NVIDIA

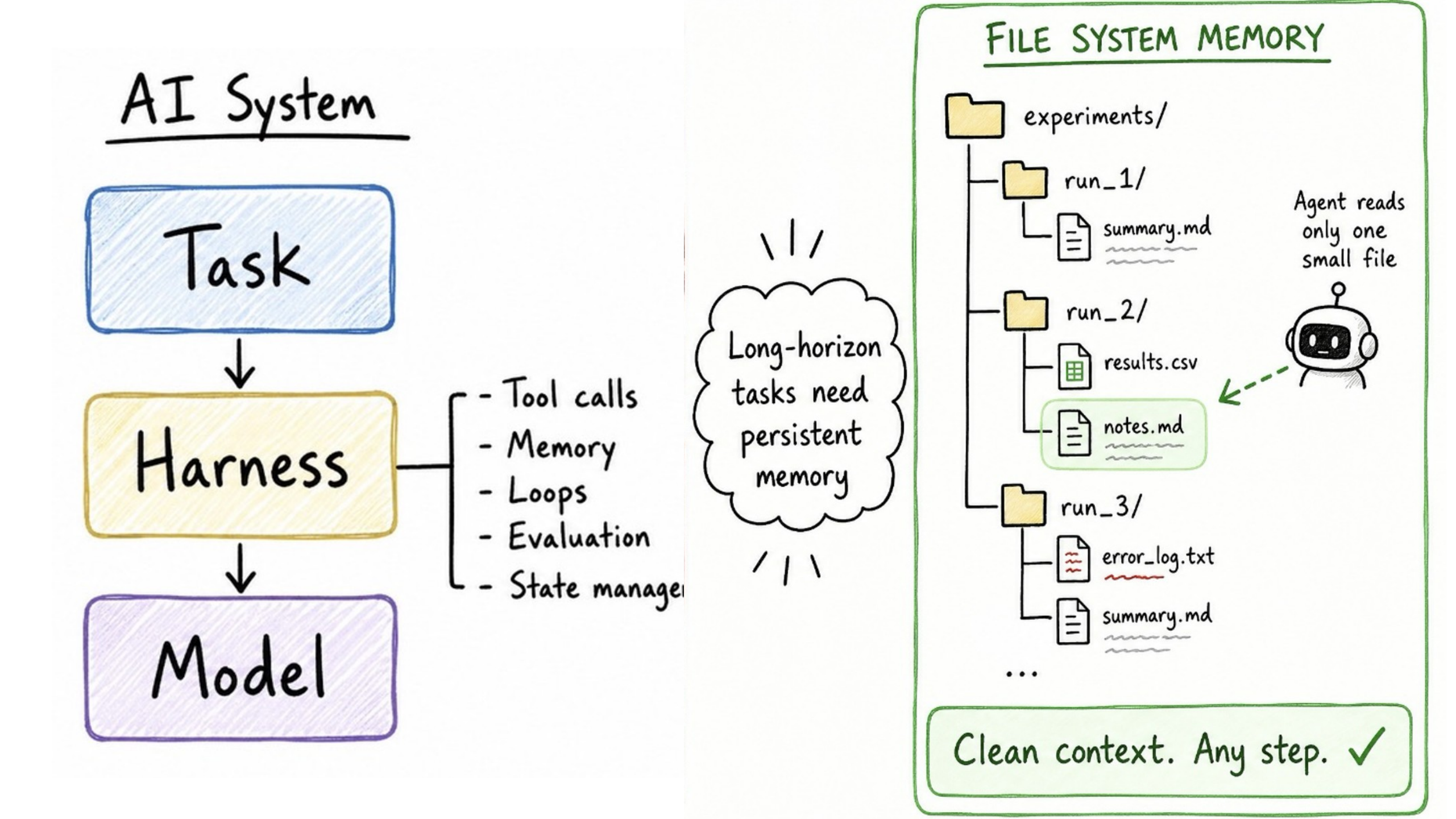
The Anatomy of an Agent Harness

the harness gives the mind a body



perceive → decide → act → observe → repeat

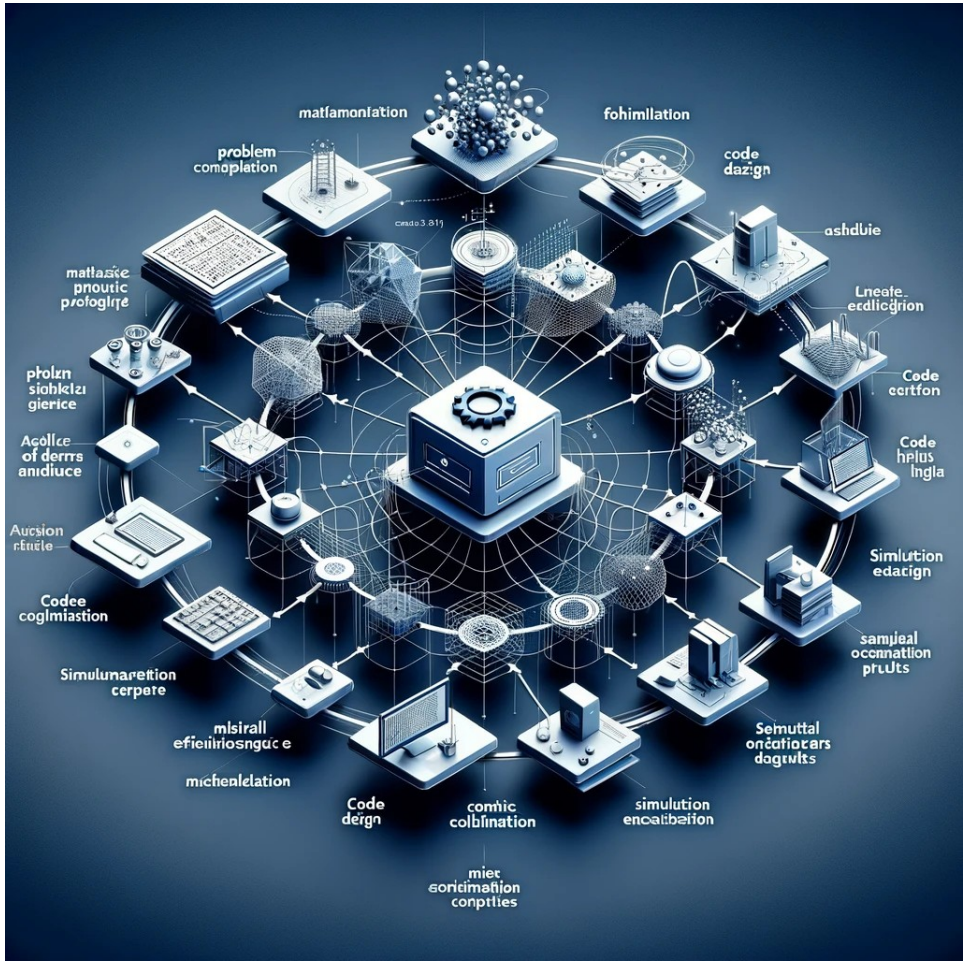
The model only ever *decides*. Everything else is harness.


AI System
Task
Harness
Model
- Tool calls
- Memory
- Loops
- Evaluation
- State manage
Long-horizon tasks need persistent memory
FILE SYSTEM MEMORY
experiments/
run_1/
summary.md
run_2/
results.csv
notes.md
run_3/
error_log.txt
summary.md
...
Agent reads only one small file
Clean context. Any step. ✓

What it might take to get to a **“scientific superintelligence”** that can run for days or weeks without intervention, master the literature, design instruments, direct large robotic facilities, and originate new theories.

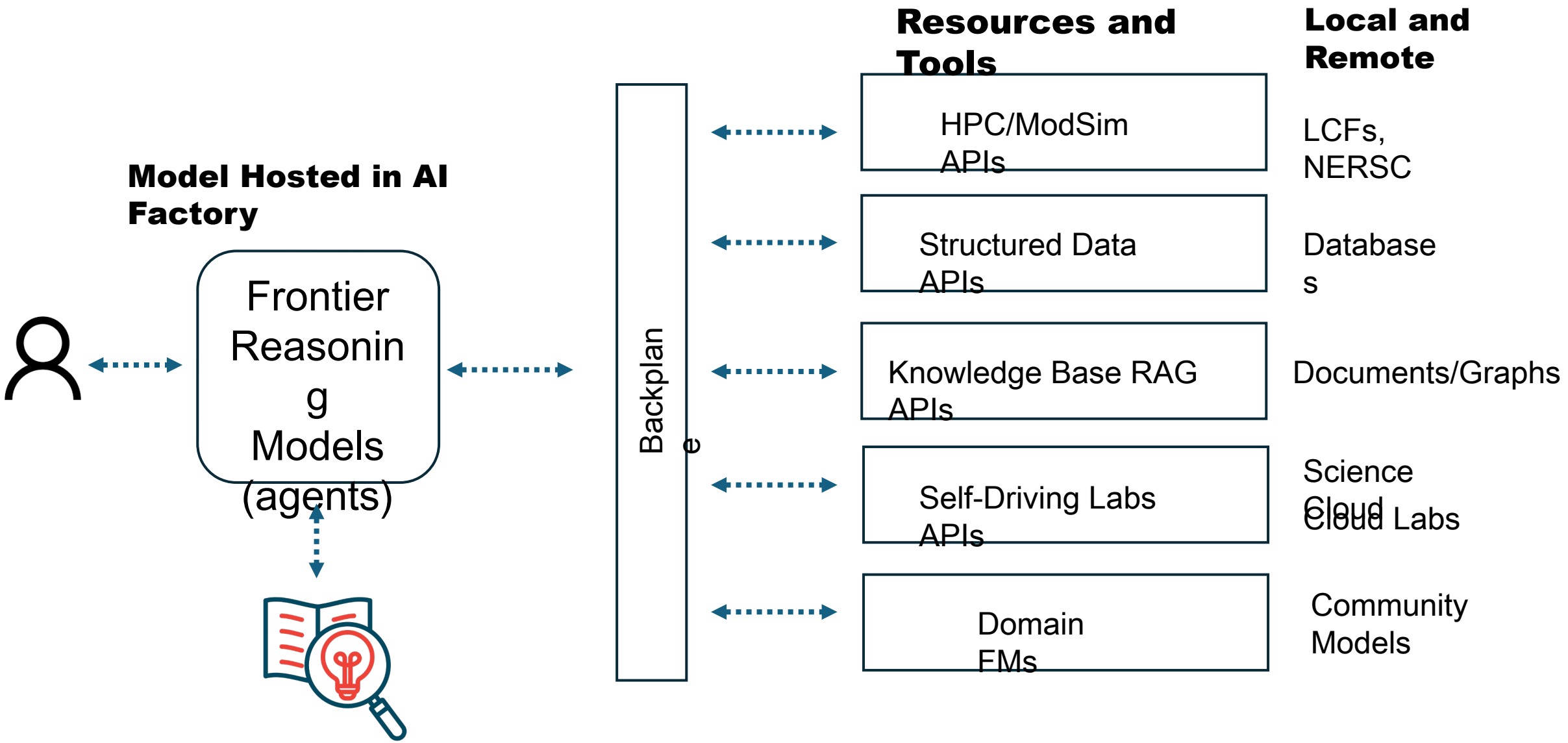
AI Assistant for “Computational” Science

- Open Problem
- Mathematical Formulation
- Numerical or Symbolic Formulation
- Efficient Algorithm
- Parallel HPC implementation
- Simulation Campaign at Exascale HPC
- Analysis of Output
- Insights and Closing the Loop



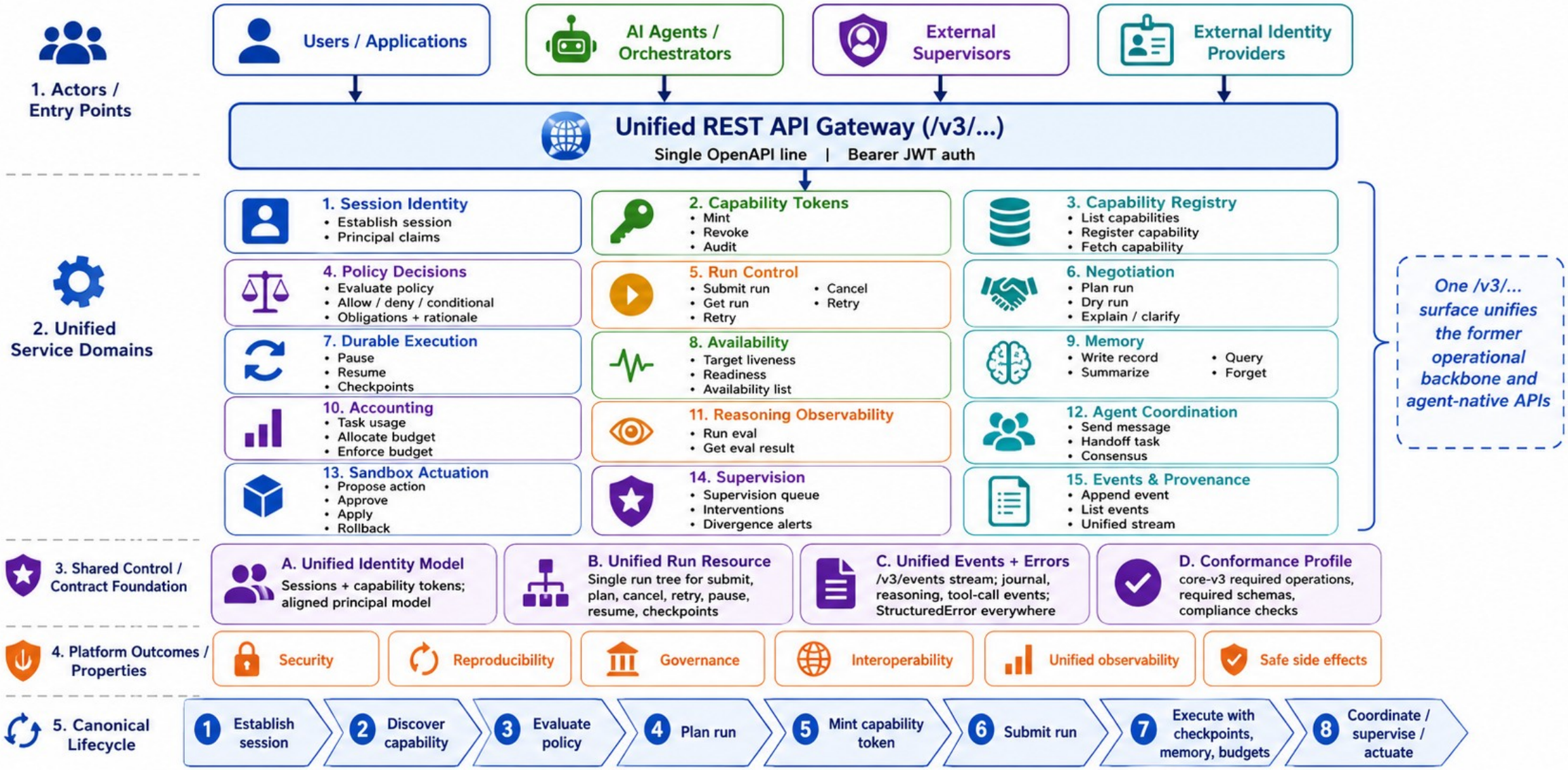
GPT4 Notion of what AI assistant Looks like

Scientific AI Platform Example



Genesis Mission Platform API Architecture

Unified core contract surface under /v3/... defined by gmp-core-v3.yaml



We define eight capabilities that we want agents to take on

Each capability is a job a human researcher does today; the roadmap says how much of it an agent takes on, and when.

Capability	Today a researcher…	The agent will…
Research (A1)	Spends weeks reading papers and hunting for datasets	Performs that survey in hours, across papers and DOE data, every claim traceable to its source
Propose ideas (A2)	Draws on experience to guess what might work	Generates candidate hypotheses, ranked by novelty and feasibility
Design experiment (A3)	Plans tests by intuition and habit	Picks the tests that teach the most per dollar and per hour of facility time
Run experiment (A4)	Queues jobs, books beamtime, babysits runs	Drives supercomputers, light sources, and robotic labs directly, within approved limits
Analyze results (A5)	Analyzes data by hand, tool by tool	Compares prediction to measurement with honest error bars; feeds the answer into the next idea
Keep campaign going (A6)	Manages one project at a time	Orchestrates many investigations at once, for weeks, within budget, recovering from failures
Keep records (A7)	Keeps lab notebooks, unevenly	Records every action and the reasoning behind it, so any result can be audited and reproduced
Stay accountable (A8)	—	Gives scientists the controls: what agents may do alone, what needs sign-off, when to raise the bar